FlexQL

A High-Performance In-Memory Database Engine

Design Document

Author	K chaitanya
Repository	github.com/devchaitanya/FlexQl
Language	C++17 (no external DB libraries)
Platform	Linux (GCC 15 / Clang 20)

Design Lab — Database Systems Project

0 Contents

1 System Overview **3**

2 How Data is Stored **3**
2.1 Columnar In-Memory Layout . . . 3
2.2 StringArena Slab Allocator . . . 4
2.3 Why This Design? . . . 4

3 Indexing Method **4**
3.1 Flat Open-Addressing Hash Map (Primary Key only) . . . 4
3.2 Why Not `std::unordered_map`? . . . 5
3.3 Tables Without a Primary Key . . . 5

4 Caching Strategy **5**
4.1 LRU Query Cache . . . 5
4.2 Thread Safety . . . 5
4.3 Why LRU over LFU? . . . 5

5 Handling of Expiration Timestamps **6**
5.1 Insertion . . . 6
5.2 Filtering . . . 6
5.3 Physical Removal (Compaction) . . . 6

6 Multithreading Design **6**
6.1 Thread Pool . . . 6
6.2 Per-Table `shared_mutex` . . . 6
6.3 WAL Writer . . . 7
6.4 TCP_NODELAY . . . 7

7 Persistence — WAL and Snapshots **7**
7.1 Write-Ahead Log (WAL) . . . 7
7.2 Binary Snapshots . . . 7
7.3 Recovery on Startup . . . 8

8 SQL Language Support **8**
8.1 DDL . . . 8
8.2 DML . . . 8
8.3 DQL . . . 8
8.4 WHERE Clause (Single Condition) . . . 8
8.5 Utility Commands . . . 9

9 Wire Protocol **9**

10 Other Design Decisions **9**
10.1 Parser Fast Path for INSERT . . . 9
10.2 Compiler and Build Optimizations . . . 9
10.3 SIMD-Accelerated Numeric Scans (AVX2) . . . 10
10.4 PK Fast Paths for DELETE, UPDATE, and SELECT IN . . . 10
10.5 DoS Protection on `recv_sql` . . . 10

10.6 JOIN Deadlock Prevention . . . 11
10.7 Database Singleton . . . 11
10.8 Single-Condition WHERE Enforcement . . . 11

11 Performance Results **11**
11.1 INSERT Throughput . . . 11
11.2 SELECT Queries . . . 12
11.3 Aggregate Queries . . . 12
11.4 JOIN Queries . . . 12
11.5 UPDATE and DELETE . . . 13
11.6 LRU Cache Effect . . . 13
11.7 Concurrency — Concurrent Reads . . . 13
11.8 Concurrency — Mixed Read/Write Workload . . . 13
11.9 Concurrency — Concurrent Inserts (Separate Tables) . . . 14
11.10 Memory Footprint . . . 14
11.11 Unit Test Results . . . 14

12 Compilation and Execution **14**
12.1 Requirements . . . 14
12.2 Build . . . 15
12.3 Run the Server . . . 15
12.4 Run the Interactive Client (REPL) . . . 15
12.5 Run the Benchmark . . . 15
12.6 Source Layout . . . 15

1 System Overview

FlexQL is a client-server, in-memory relational database implemented entirely in C++17 with no external database libraries. It exposes a SQLite-compatible C API and a TCP wire protocol that allows multiple concurrent clients.

1.0 Architecture Diagram

```
Client (REPL / benchmark)
        |  TCP socket  (raw SQL text terminated by ';')
        v
 flexql-server
  +-- Thread Pool  (N = hardware_concurrency workers)
  +-- Parser  ->  Executor
  +-- Table Storage  (columnar StringArena + flat arrays)
  +-- LRU Query Cache
  +-- PK Index  (flat open-addressing FNV-64 hash map)
  +-- WAL Writer  (zero-copy writev + SSE4.2 CRC32c)
  +-- Snapshot Engine  (binary checkpoints)
```

High-level architecture

1.0 Client API

The client library (`flexql.h`) exposes exactly four functions as prescribed:

```
typedef struct FlexQL FlexQL;
#define FLEXQL_OK    0
#define FLEXQL_ERROR 1

int  flexql_open (const char *host, int port, FlexQL **db);
int  flexql_close(FlexQL *db);
int  flexql_exec (FlexQL *db, const char *sql,
                  int (*callback)(void*,int,char**,char**),
                  void *arg, char **errmsg);
void flexql_free (void *ptr);
```

The callback is invoked once per result row: `callback(data, columnCount, values[], columnNames[])`.

2 How Data is Stored

2.1 Columnar In-Memory Layout

Each table uses two parallel flat arrays rather than a `vector<vector<string>>`:

```
StringArena                    arena_;     // slab allocator owns
   all string bytes
std::vector<RowMeta>           rows_meta_;  // {expires_at,
   deleted} per row
std::vector<std::string_view> rows_vals_;  // rows * cols
   string_views into arena
```

```
int                                 num_cols_;
```

Value at row r, column c: `rows_vals_[r * num_cols_ + c]`

2.2 StringArena Slab Allocator

```
class StringArena {
    static constexpr size_t BLOCK = 4 * 1024 * 1024;  // 4 MB
       slabs
    std::vector<std::unique_ptr<char[]>> blocks_;
    size_t used_ = BLOCK;
public:
    std::string_view intern(std::string_view sv) {
        if (sv.size() > BLOCK) { /* dedicated oversized block */
            }
        if (used_ + sv.size() > BLOCK) { /* new slab */ }
        char* p = blocks_.back().get() + used_;
        std::memcpy(p, sv.data(), sv.size());
        used_ += sv.size();
        return {p, sv.size()};
    }
};
```

Slab blocks never move, so interned `string_view` references are **stable for the lifetime of the table**.

2.3 Why This Design?

Design choice	Alternative	Reason
Flat `string_view` array	`vector<vector<string>>`	Eliminates 1 heap allocation per row. For 1 M rows: zero per-row `malloc` calls
StringArena slabs	Per-string `std::string`	All data in contiguous 4 MB blocks; hot strings stay in L2/L3 cache
Pre-reserve on first batch	Default `push_back` growth	Avoids 7+ reallocation spikes during a 1 M-row insert benchmark

Table 1: Storage design choices and rationale

3 Indexing Method

3.1 Flat Open-Addressing Hash Map (Primary Key only)

`PrimaryIndex` in `include/index/pk_index.h`:

- **Hash function:** FNV-64 — fast, good distribution for string keys
- **Collision resolution:** linear probing — cache-friendly, no pointer chasing
- **Load factor threshold:** 0.6 — rehash doubles capacity when exceeded
- **Keys:** `string_view` into arena (zero-copy lookup)

- **Supports:** `insert`, `lookup`, `remove`, `update`, `clear`

```
static uint64_t fnv64(std::string_view s) noexcept {
    uint64_t h = 14695981039346656037ULL;
    for (unsigned char c : s) h = (h ^ c) * 1099511628211ULL;
    return h;
}
// lookup: probe from fnv64(key) % capacity until match or empty
    slot
```

FNV-64 hash + probe loop

3.2 Why Not `std::unordered_map`?

Each `unordered_map` bucket is a linked list — pointer-chasing across cache lines. For 1 M rows, the flat open-addressing map is approximately 3× faster on lookups because all slots are contiguous in memory.

3.3 Tables Without a Primary Key

Tables without a `PRIMARY KEY` constraint have no index. All queries do a linear scan of `rows_meta_` and `rows_vals_`. This is intentional — the benchmark table (`BIG_USERS`) has no primary key, so the hot insert path pays zero index overhead.

4 Caching Strategy

4.1 LRU Query Cache

- **Policy:** Least Recently Used (LRU)
- **Capacity:** Configurable via `cache_capacity` in `config/server.conf` (default 1024 entries)
- **Key format:** `TABLE|cols|W:col=op=val|O:colDA|D|Lnn`
- **Invalidation:** Any `INSERT` / `UPDATE` / `DELETE` / `TRUNCATE` calls `invalidate_prefix(table_n` evicting all cached results for that table
- **Scope:** `SELECT` queries only; compound `WHERE` predicates bypass the cache (they are not fully serialised to a key)

4.2 Thread Safety

The cache is protected by a `shared_mutex`: multiple readers acquire a shared lock concurrently; a write (`put` / `invalidate`) takes an exclusive lock.

4.3 Why LRU over LFU?

The benchmark pattern is: insert 1 M rows once, then run repeated `SELECT` scans. LRU captures the *"recently scanned table"* pattern with no frequency bookkeeping overhead. LFU would yield identical hit rates but add per-entry counter updates on every access.

5 Handling of Expiration Timestamps

5.1 Insertion

Every row stores `int64_t expires_at` (Unix epoch seconds; `0` = never expires). The benchmark sends rows with a 5th column as the TTL value. The executor detects a column-count mismatch of +1, extracts the last value as `expires_at`, and strips it before passing the row to storage.

5.2 Filtering

```
bool Table::is_expired(const RowMeta& m) const {
    if (m.expires_at == 0) return false;
    return m.expires_at <= (int64_t)std::time(nullptr);
}
```

Every scan, delete, and update skips expired rows. They become invisible to all queries immediately upon expiry — no background job required for correctness.

5.3 Physical Removal (Compaction)

A background **TTL sweeper** thread (`TTLManager`) runs every `ttl_sweep_interval` seconds (default 60 s). It calls `Table::compact()` on each table, which rebuilds both parallel arrays and the PK index, physically removing rows where `deleted == true` or `is_expired() == true`.

This two-phase design (logical deletion → background physical removal) avoids blocking the hot INSERT/SELECT path with compaction work.

6 Multithreading Design

6.1 Thread Pool

`ThreadPool` creates `hardware_concurrency()` worker threads (configurable via `threads` in `server.conf`). Each accepted TCP connection is dispatched to a free worker via a `std::queue` protected by `std::mutex` + `std::condition_variable`.

6.2 Per-Table `shared_mutex`

Operation	Lock type	Behaviour
`scan()`	`shared_lock`	Multiple concurrent readers
`insert_flat()` / `insert_batch()`	`unique_lock`	Exclusive write
`delete_rows()` / `update_rows()`	`unique_lock`	Exclusive write
`compact()`	`unique_lock`	Background only

Table 2: Locking strategy per table operation

Multiple clients can read the same table simultaneously. A write from one client blocks only that table, not others.

6.3 WAL Writer

The WAL writer uses a single `std::mutex`. `writev` takes ∼57 µs per call; contention is negligible under the benchmark's sequential insert pattern.

6.4 TCP_NODELAY

All accepted sockets have `TCP_NODELAY` set, disabling Nagle's algorithm. This eliminates up to 40 ms of artificial per-request latency on loopback interfaces.

7 Persistence — WAL and Snapshots

7.1 Write-Ahead Log (WAL)

Every mutation (`INSERT`, `UPDATE`, `DELETE`, `TRUNCATE`, `CREATE TABLE`, `DROP TABLE`) is written to the WAL before execution. On crash and restart, the WAL is replayed to recover all committed writes.

7.1.0 Record Format (20-byte header + SQL payload)

```
[ 4B magic ][ 8B LSN ][ 4B payload_len ][ 4B CRC32c ][ SQL bytes
    ]
```

7.1.0 Zero-Copy Append

```
uint8_t hdr[20];    // built on stack (magic, LSN, len, checksum)
struct iovec iov[2] = {{hdr, 20}, {sql.data(), sql.size()}};
::writev(fd_, iov, 2);    // single syscall, no intermediate copy
```

7.1.0 Key Design Decisions

- `fdatasync` is called *only* at shutdown/snapshot — not per write. This saves ∼1300 ms over the benchmark (100 × 13 ms per flush).
- **CRC32c** uses the SSE4.2 hardware instruction `_mm_crc32_u64`: ∼9 ms for 200 × 275 KB batches vs ∼166 ms for software lookup.

7.2 Binary Snapshots

Checkpoints are written to `data/snapshots/snap_`*`lsn`*`.bin`:

```
[ 8B magic ][ 8B snapshot_lsn ]
[ for each table:
    4B name_len, name bytes
    4B num_columns, [ col_name, col_type, pk, not_null ] * N
    8B num_rows,    [ col_values..., expires_at ] * M
]
```

7.3 Recovery on Startup

1. Load the snapshot with the highest LSN (restores all schemas and row data)
2. Replay WAL records with `LSN > snapshot_lsn` (recovers mutations since checkpoint)
3. Accept client connections

8 SQL Language Support

The parser is a hand-written recursive-descent parser with a dedicated fast path (`fast_parse_insert`) for performance-critical INSERT statements.

8.1 DDL

```
CREATE TABLE [IF NOT EXISTS] t (col type [PRIMARY KEY] [NOT NULL
   ], ...)
  -- types: INT, DECIMAL, VARCHAR(n), TEXT, DATETIME
DROP TABLE [IF EXISTS] t
TRUNCATE TABLE t
ALTER TABLE t ADD COLUMN col type
```

8.2 DML

```
INSERT INTO t [(col, ...)] VALUES (v, ...) [, (v, ...) ...]
UPDATE t SET col=v [, col=v ...] [WHERE ...]
DELETE FROM t [WHERE ...]
```

8.3 DQL

```
SELECT [DISTINCT] col [AS alias], ...
  FROM t
  [INNER JOIN t2 ON t.col = t2.col]
  [LEFT  JOIN t2 ON t.col = t2.col]
  [WHERE ...]
  [GROUP BY col [, col ...] [HAVING ...]]
  [ORDER BY col [ASC|DESC] [, col [ASC|DESC] ...]]
  [LIMIT n] [OFFSET m]

SELECT COUNT(*) | SUM(col) | AVG(col) | MIN(col) | MAX(col)
  FROM t [WHERE ...]
```

8.4 WHERE Clause (Single Condition)

As specified in the assignment, only a single condition is supported per `WHERE` clause. Logical combinations (`AND`, `OR`, `NOT`) are explicitly rejected by the parser.

Supported operators: `=`, `!=`, `<`, `<=`, `>`, `>=`, `LIKE` (`%` and `_` wildcards), `BETWEEN v AND v`, `IN (v, ...)`, `IS NULL`, `IS NOT NULL`.

Each `WHERE` predicate is represented as a single `Predicate::LEAF` node evaluated during the table scan.

8.5 Utility Commands

```sql
SHOW TABLES
SHOW DATABASES
SHOW COLUMNS FROM t     -- synonym for DESCRIBE
DESCRIBE t
USE [DATABASE] name
```

9 Wire Protocol

Plain-text, newline-delimited over TCP. The client sends raw SQL terminated by `;` in a single `send` call. The server replies:

```
# SELECT with results:
COLS col1 col2 col3
ROW  val1 val2 val3
...
END

# Non-SELECT (INSERT, UPDATE, ...):
OK
END

# Error:
ERROR: <message>
END
```

The `COLS` header carries real column names including `AS` aliases. The benchmark client (`bench/flexql.cpp`) ignores non-`ROW` lines, so the header is fully backward-compatible.

Buffering: The server uses a 64 KB `recv` buffer (pre-reserved to 512 KB for large `INSERT` batches). The client uses a 4 KB local read buffer to eliminate per-character `recv` syscalls.

10 Other Design Decisions

10.1 Parser Fast Path for INSERT

The parser has two code paths: a full recursive-descent parser for all SQL statements, and a dedicated `fast_parse_insert()` for INSERT. The fast path operates as a single-pass character scanner over the raw SQL buffer, producing `std::string_view`s directly into the input — zero heap allocations for a 25K-row batch. This avoids constructing 25K `Token` objects and 25K inner vectors that the general parser would create.

10.2 Compiler and Build Optimizations

The Makefile uses aggressive optimization flags:

- `-O3 -march=native`: enables auto-vectorisation and targets the host CPU's instruction set (AVX2, SSE4.2)
- `-funroll-loops`: reduces loop overhead in tight scan loops

- `-flto=auto`: link-time optimisation across translation units, allowing cross-file inlining of hot functions like `StringArena::intern()`

No external libraries are linked. The only hardware-specific dependency is SSE4.2 for the CRC32c instruction in the WAL writer and AVX2 for the SIMD-accelerated scan intrinsics.

10.3 SIMD-Accelerated Numeric Scans (AVX2)

Every `INT` and `DECIMAL` column maintains a parallel `vector<int64_t>` alongside the `string_view` storage. On insert, each numeric value is parsed once with an allocation-free integer parser (`parse_int64`) and appended to `int_cols_[col]`. This dual representation eliminates the `std::stod(std::string(string_view))` call that otherwise dominates scan cost.

For simple numeric `WHERE` predicates, the scan loop dispatches to hand-written AVX2 intrinsics that compare 4 × `int64` values per CPU cycle:

```
__m256i vals = _mm256_loadu_si256(col + i);    // load 4 int64s
__m256i mask = _mm256_cmpgt_epi64(vals, tgt); // compare all 4
int m = _mm256_movemask_pd(cast(mask));         // 4-bit result
if (m & 1) if (alive(i))   out.push_back(i);
if (m & 2) if (alive(i+1)) out.push_back(i+1);
// ...
```

AVX2 scan kernel (simplified)

Dedicated AVX2 kernels handle `=`, `!=`, `>`, `>=`, `<`, `<=`, `BETWEEN`, and `IN`. Non-numeric predicates (`LIKE`, `IS NULL`, `VARCHAR` columns) fall through to the original per-row `compare_values()` path.

`ORDER BY` on numeric columns also uses the native `int64_t` arrays, eliminating `stod()` from every sort comparison.

Memory overhead: one `int64_t` (8 bytes) per numeric value per row. For a 1 M-row table with 3 numeric columns, this adds ∼24 MB (162 MB total vs. 139 MB without).

10.4 PK Fast Paths for DELETE, UPDATE, and SELECT IN

The hash-map index is now used for more than just `SELECT WHERE pk = X`:

- `DELETE WHERE pk = X`: O(1) hash lookup + tombstone, instead of O(n) scan
- `UPDATE WHERE pk = X`: O(1) hash lookup + in-place modification
- `SELECT/DELETE/UPDATE WHERE pk IN (v1,...,vk)`: O(k) hash lookups

This makes point mutations as fast as point reads.

10.5 DoS Protection on `recv_sql`

The server enforces a 16 MB cap on incoming SQL messages. If a client sends a message exceeding this limit, `recv_sql()` returns an empty string and the connection is closed. This prevents a malicious or buggy client from exhausting server memory with a single unbounded request.

10.6 JOIN Deadlock Prevention

When executing a JOIN, the executor must lock two tables. To prevent ABBA deadlocks when two concurrent queries join the same tables in different order, the executor always acquires table locks in **alphabetical order** of the table name. This total ordering guarantees deadlock freedom without a lock manager.

10.7 Database Singleton

The `Database` class uses the singleton pattern (`Database::instance()`) to provide a single global table registry. All threads access the same instance; the registry itself is protected by a `shared_mutex` for concurrent table lookups. Individual table mutations are protected by per-table locks (see Section 6).

10.8 Single-Condition WHERE Enforcement

The assignment restricts `WHERE` to a single condition — no `AND`, `OR`, or `NOT` combinators are allowed. After parsing one predicate atom, the parser checks whether the next token is `AND` or `OR` and raises a `ParseError` if so. This ensures the grading scripts receive deterministic, single-condition behaviour. Operators like `BETWEEN ... AND`, `IN`, and `LIKE` are unaffected because their `AND` token is part of the operator syntax, not a logical combinator.

11 Performance Results

All benchmarks run on a 1 M-row table (`BENCH`: 5 columns, no primary key) unless stated otherwise.

Hardware: Intel Core i5-1135G7 @ 2.40 GHz, 14 GB RAM, Ubuntu Linux.
Build: `-O3 -march=native -std=c++17`.

11.1 INSERT Throughput

Dataset	Elapsed	Throughput
1 M rows	277 ms	3,610,108 rows/sec
10 M rows	3,548 ms	2,818,489 rows/sec

Table 3: INSERT benchmark — batch size 5,000 (with dual int64 storage)

The dual `int64_t` storage adds a single `parse_int64()` call per numeric value during INSERT. This overhead is negligible: throughput remains above 2.8 M rows/sec even at 10 M rows.

11.2 SELECT Queries

Query	Rows	Elapsed
`SELECT COUNT(*) FROM BENCH`	1	174 ms
`SELECT * FROM BENCH` (full scan)	1 M	1,504 ms
`SELECT * WHERE ID = 500000`	1	0.75 ms
`SELECT * WHERE SALARY > 90000`	139,986	172 ms
`SELECT * WHERE SALARY BETWEEN 50000 AND 60000`	140,015	168 ms
`SELECT * WHERE DEPT IN ('ENG','HR','FIN')`	375,000	571 ms
`SELECT * WHERE NAME LIKE 'user1%'`	111,112	179 ms
`SELECT * ORDER BY SALARY DESC LIMIT 10`	10	200 ms
`SELECT * ORDER BY ID DESC LIMIT 100 OFFSET 1000`	100	209 ms
`SELECT DISTINCT DEPT FROM BENCH`	8	94 ms

Table 4: SELECT query latencies on 1 M-row table

SIMD-accelerated numeric scans (`WHERE ID = 500000`, `SALARY > 90000`, `BETWEEN`) run 2−−148× faster than the pre-SIMD string-parsing path. `ORDER BY` on numeric columns uses native `int64_t` comparisons, reducing sort time from ∼1,100 ms to ∼200 ms (5.7×).

11.3 Aggregate Queries

Query	Elapsed
`SELECT SUM(SALARY)`	165 ms
`SELECT AVG(SALARY)`	174 ms
`SELECT MIN(SALARY)`	173 ms
`SELECT MAX(SALARY)`	173 ms
`GROUP BY DEPT` (8 groups)	226 ms
`GROUP BY DEPT HAVING AVG(SALARY)>60000`	174 ms

Table 5: Aggregate query latencies on 1 M-row table

11.4 JOIN Queries

Query	Rows	Elapsed
`INNER JOIN` (1 M × 8, all match)	1 M	960 ms
`INNER JOIN + WHERE SALARY>90000`	139,986	372 ms
`LEFT JOIN` (1 M × 8)	1 M	1,195 ms

Table 6: JOIN query latencies (BENCH 1 M rows × DEPARTMENTS 8 rows)

11.5 UPDATE and DELETE

Operation	Elapsed
`UPDATE` single row (`WHERE ID=1`)	91 ms
`UPDATE` 1,000 rows (`WHERE ID<=1000`)	74 ms
`UPDATE` filtered (`WHERE DEPT='QA'`)	55 ms
`DELETE` 100 rows (`WHERE ID<=100`)	76 ms
`DELETE` filtered (`WHERE DEPT='LEGAL'`)	54 ms

Table 7: UPDATE / DELETE latencies on 1 M-row table

11.6 LRU Cache Effect

Invocation	Elapsed
First (cold cache)	0.80 ms
Second (warm cache)	0.10 ms
Third (warm cache)	0.03 ms

Table 8: Cache hit vs. miss for a point query. Cold query now uses SIMD fast path (0.8 ms) instead of `stod()`-based scan (∼113 ms). Speedup: ∼29×

11.7 Concurrency — Concurrent Reads

Threads	Queries	Avg Latency	COUNT(*) wall-clock
1	20	0.68 ms	144 ms
4	80	0.63 ms	201 ms
8	160	0.46 ms	406 ms

Table 9: Concurrent read scaling (`shared_lock` allows parallel readers)

Per-query latency is sub-millisecond because the SIMD numeric scan path processes point queries in ∼0.5 ms. Wall-clock for 8 simultaneous `COUNT(*)` scans is only 2.8× that of a single scan, showing good read parallelism.

11.8 Concurrency — Mixed Read/Write Workload

Config	Read ops	Write ops	Total ops/sec
4 readers + 1 writer	853	63	305
4 readers + 4 writers	1,099	67	370

Table 10: Mixed workload over 3 seconds (point queries + single-row UPDATEs)

With SIMD scans, read throughput jumped from ∼33 to ∼305 ops/sec (9×) because point queries now complete in sub-millisecond time instead of ∼100 ms. Write contention

still reduces read throughput because each `UPDATE` acquires an exclusive `unique_lock`, blocking concurrent readers momentarily.

11.9 Concurrency — Concurrent Inserts (Separate Tables)

Threads	Total Rows	Wall-clock	Agg. Throughput
1	50 K	10 ms	5.00 M rows/sec
4	200 K	13 ms	15.38 M rows/sec
8	400 K	30 ms	13.33 M rows/sec

Table 11: Concurrent INSERT into separate tables (per-table locks, no contention)

When each thread writes to its own table, per-table `unique_lock` causes zero contention. Aggregate throughput scales almost linearly: 8 threads achieve 2.6× the throughput of 1 thread.

11.10 Memory Footprint

State	Server RSS
Idle (no tables)	7.1 MB
After 1 M-row seed (5 columns)	161.8 MB
After full benchmark suite	∼1.0 GB

Table 12: Server resident set size at different stages

The 1 M-row table uses ∼155 MB of memory (StringArena slabs + row metadata + `string_view` array + `int_cols_` native integer arrays). The extra ∼24 MB over the pre-SIMD baseline comes from the dual `int64_t` storage for the 3 numeric columns ($3 \times 1M \times 8B = 24MB$).

11.11 Unit Test Results

564 / 564 standalone tests pass (`make test`). Test categories: parser (17 functions), primary-key index (3), StringArena (4), table storage (10), executor end-to-end (6), concurrency stress (3), adversarial inputs (5). Sanitiser targets `make test-asan` and `make test-tsan` are also available.

12 Compilation and Execution

12.1 Requirements

- GCC 11+ or Clang 14+ with C++17 support
- Linux (uses `writev`, `shared_mutex`, POSIX threads)
- No external libraries required

12.2 Build

```
git clone https://github.com/devchaitanya/FlexQl.git
cd FlexQl
make all -j$(nproc)
# Produces: bin/flexql-server  bin/flexql-client
```

12.3 Run the Server

```
mkdir -p data/wal data/snapshots
./bin/flexql-server 9000
# Configuration: config/server.conf
```

12.4 Run the Interactive Client (REPL)

```
./bin/flexql-client 127.0.0.1 9000
```

```
flexql> CREATE TABLE users (id INT PRIMARY KEY NOT NULL, name
   VARCHAR(64));
flexql> INSERT INTO users VALUES (1, 'Alice', 0), (2, 'Bob', 0);
flexql> SELECT * FROM users ORDER BY name;
+----+-------+
| id | name  |
+----+-------+
| 1  | Alice |
| 2  | Bob   |
+----+-------+
2 rows in set (0.000 sec)
flexql> .exit
```

12.5 Run the Benchmark

```
./bench/benchmark 1000000      # 1M-row insert
./bench/benchmark 10000000     # 10M rows
./bench/benchmark 20000000     # 20M rows
./bench/benchmark --unit-test # unit tests only
```

12.6 Source Layout

```
FlexQl/
  include/          -- public headers (grouped by subsystem)
    cache/          -- lru_cache.h
    common/         -- types.h
    concurrency/    -- thread_pool.h
    expiration/     -- ttl_manager.h
    index/          -- pk_index.h
    network/        -- protocol.h, server.h
    parser/         -- ast.h, parser.h, token.h
    query/          -- executor.h
```

```
    storage/         -- arena.h, database.h, snapshot.h, table.h,
        wal.h
    utils/           -- string_utils.h
  src/               -- implementation (.cpp files, same layout
     as include/)
    client/          -- flexql.cpp (API), client.cpp (REPL)
  bench/             -- benchmark_flexql.cpp, flexql.h (
     instructor-provided)
  config/            -- server.conf
  docs/              -- design.tex  (this document)
  data/              -- wal/ and snapshots/ (created at runtime)
  Makefile
  README.md
  DESIGN.md
```